HW12-PartB_Adam_Nelson-Archer

April 24, 2026

1 HW12 Part B :: Semantic Segmentation in Videos

COSC 6373 - Adam Nelson-Archer, 2140122

1.1 Setup

On Linux, if the notebook uses the system Python, automatic install may be blocked (PEP 668). In that case create a virtual environment, run `pip install torch torchvision` inside it, install `ipykernel`, pick that environment as the Jupyter kernel, and re-run from the top.

[22]:
```
import subprocess
import sys

try:
    import torch
except ModuleNotFoundError:
    subprocess.check_call(
        [sys.executable, "-m", "pip", "install", "torch", "torchvision"]
    )
    try:
        import torch
    except ModuleNotFoundError as e:
        raise RuntimeError(
            "PyTorch is still not importable. Use a venv for this project,␣
↪install torch and torchvision there, "
            "select that interpreter as the notebook kernel, then run all cells␣
↪again. "
            "Example: python3 -m venv .venv  then  .venv/bin/pip install torch␣
↪torchvision ipykernel"
        ) from e

import time
import cv2
import numpy as np
import matplotlib.pyplot as plt
from torchvision import models, transforms
from urllib.request import Request, urlopen
from pathlib import Path
```

```
import warnings

warnings.filterwarnings("ignore")

print("OpenCV:", cv2.__version__)
print("PyTorch:", torch.__version__)
```

```
OpenCV: 4.13.0
PyTorch: 2.11.0+cu130
```

1.2 Question 1

Are image segmentation models considered multi-task architectures, and what does "multi-task" mean in this context?

In the usual deep learning sense, multi-task learning means a single network is trained to solve more than one distinct supervised problem with separate heads and loss terms, for example joint depth estimation and semantic labeling, or detection plus segmentation. A standard semantic segmentation model is more often described as a single task: it performs dense per-pixel classification into K classes, which is multi-class, not multiple tasks. The pre-trained models used here are trained to produce one class map (for 21 Pascal VOC style labels including background) per frame. That is one structured output for one objective.

HW12 Part A focused on unsupervised pixel grouping (K-means and Mean-Shift) without class labels, while this part uses supervised semantic segmentation on MobileNetV3 backbones with dedicated segmentation decoders and pre-trained weights. That contrast is about supervision and objective, not about multi-task learning.

1.3 Question 2

Video file. The code below uses `sample_video_240x480.mp4` in the local part B folder, downscaled from a SampleLib clip (sample-5s.mp4). That clip is short and small enough to satisfy the 5-10 second, 240x480 requirement. The SampleLib page states their sample clips are for demonstration; keep any other clip you substitute compliant with its license and cite the source in your report.

OpenCV `VideoCapture` and `VideoWriter` use the standard open, read in a loop, and release pattern for each frame.

[23]:
```
# Load video
VIDEO_NAME = "sample_video_240x480.mp4"
VIDEO_PATH = Path(VIDEO_NAME)

if not VIDEO_PATH.exists():
    # Download a sample low-res video if not present
    video_url = "https://download.samplelib.com/mp4/sample-5s.mp4"
    req = Request(video_url, headers={'User-Agent': 'Mozilla/5.0'})
    with urlopen(req) as response, open(VIDEO_PATH, 'wb') as out_file:
        out_file.write(response.read())
    print(f"Downloaded sample video to {VIDEO_PATH.resolve()}")
else:
```

```
    print(f"Video already exists at {VIDEO_PATH.resolve()}")

tmp_cap = cv2.VideoCapture(str(VIDEO_PATH))
vw = int(tmp_cap.get(cv2.CAP_PROP_FRAME_WIDTH))
vh = int(tmp_cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
vfps = float(tmp_cap.get(cv2.CAP_PROP_FPS)) or 25.0
vframes = int(tmp_cap.get(cv2.CAP_PROP_FRAME_COUNT))
tmp_cap.release()
print(f"Input video: {vw}x{vh}, {vfps:.2f} fps, {vframes} frames (all frames␣
↪are resized to 240x480 for the models)")
```

```
Video already exists at
/home/pc/projects/COSC6373/HW12/Part_B/sample_video_240x480.mp4
Input video: 480x240, 30.00 fps, 171 frames (all frames are resized to 240x480
for the models)
```

1.4 Questions 3, 4, and 5

Question 3: Parse frames with OpenCV, run inference, and write output videos. Questions 4 and 5: use `deeplabv3_mobilenet_v3_large` and `lraspp_mobilenet_v3_large` with default weights from `torchvision.models.segmentation` as listed in the PyTorch vision model documentation. Input normalization and tensor layout match the pre-trained API for these builders. Class-colored visualization uses a 21-color Pascal VOC style color map.

[24]:
```
# Set device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

OUTPUT_DIR = Path("outputs")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

deeplab = models.segmentation.deeplabv3_mobilenet_v3_large(weights="DEFAULT").
↪to(device).eval()
lraspp = models.segmentation.lraspp_mobilenet_v3_large(weights="DEFAULT").
↪to(device).eval()

preprocess = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((240, 480)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

LABEL_COLOR_MAP = [
    (0, 0, 0), (128, 0, 0), (0, 128, 0), (128, 128, 0), (0, 0, 128), (128, 0,␣
↪128), (0, 128, 128),
    (128, 128, 128), (64, 0, 0), (192, 0, 0), (64, 128, 0), (192, 128, 0), (64,␣
↪0, 128), (192, 0, 128),
```

```
    (64, 128, 128), (192, 128, 128), (0, 64, 0), (128, 64, 0), (0, 192, 0),␣
↪(128, 192, 0), (0, 64, 128),
]


def labels_to_bgr(mask: np.ndarray) -> np.ndarray:
    h, w = mask.shape
    bgr = np.zeros((h, w, 3), dtype=np.uint8)
    for i, (r, g, b) in enumerate(LABEL_COLOR_MAP):
        bgr[mask == i] = (b, g, r)
    return bgr
```

```
Using device: cuda
```

[25]:
```
OUT_DL = OUTPUT_DIR / "hw12b_segmentation_deeplabv3_mobilenetv3.mp4"
OUT_LR = OUTPUT_DIR / "hw12b_segmentation_lraspp_mobilenetv3.mp4"
OUT_CMP = OUTPUT_DIR / "hw12b_segmentation_compare_original_deeplab_lraspp.mp4"
TARGET_W, TARGET_H = 480, 240

cap = cv2.VideoCapture(str(VIDEO_PATH))
fps = float(cap.get(cv2.CAP_PROP_FPS)) or 25.0
n_all = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
fourcc = cv2.VideoWriter_fourcc(*"mp4v")

w_dl = cv2.VideoWriter(str(OUT_DL), fourcc, fps, (TARGET_W, TARGET_H))
w_lr = cv2.VideoWriter(str(OUT_LR), fourcc, fps, (TARGET_W, TARGET_H))
w_cmp = cv2.VideoWriter(str(OUT_CMP), fourcc, fps, (TARGET_W * 3, TARGET_H))

if n_all < 0:
    n_all = 0
if n_all >= 4:
    sample_idx = {0, n_all // 3, (2 * n_all) // 3, n_all - 1}
elif n_all > 0:
    sample_idx = set(range(n_all))
else:
    sample_idx = {0, 1, 2, 3}
samples = []

frame_idx = 0
time_dl = 0.0
time_lr = 0.0

with torch.inference_mode():
    while cap.isOpened():
        ret, frame_bgr = cap.read()
        if not ret:
            break
```

```
        frame_small = cv2.resize(frame_bgr, (TARGET_W, TARGET_H))
        frame_rgb = cv2.cvtColor(frame_small, cv2.COLOR_BGR2RGB)
        input_tensor = preprocess(frame_rgb).unsqueeze(0).to(device)

        t0 = time.time()
        out_dl = deeplab(input_tensor)["out"][0]
        pred_dl = out_dl.argmax(0).byte().cpu().numpy()
        time_dl += time.time() - t0

        t1 = time.time()
        out_lr = lraspp(input_tensor)["out"][0]
        pred_lr = out_lr.argmax(0).byte().cpu().numpy()
        time_lr += time.time() - t1

        bgr_dl = labels_to_bgr(pred_dl)
        bgr_lr = labels_to_bgr(pred_lr)
        alpha = 0.75
        frame_dim = (frame_small.astype(np.float32) * 0.55).astype(np.uint8)
        bgr_dl_over = cv2.addWeighted(frame_dim, 1 - alpha, bgr_dl, alpha, 0)
        bgr_lr_over = cv2.addWeighted(frame_dim, 1 - alpha, bgr_lr, alpha, 0)

        w_dl.write(bgr_dl_over)
        w_lr.write(bgr_lr_over)
        w_cmp.write(np.hstack([frame_small, bgr_dl_over, bgr_lr_over]))

        if frame_idx in sample_idx:
            samples.append((frame_rgb.copy(), bgr_dl_over.copy(), bgr_lr_over.
 ↪copy()))

        frame_idx += 1

cap.release()
w_dl.release()
w_lr.release()
w_cmp.release()

print(f"Wrote {frame_idx} frames to:")
print(" ", OUT_DL.resolve())
print(" ", OUT_LR.resolve())
print(" ", OUT_CMP.resolve())

if frame_idx > 0:
    print("\nAverage Inference FPS (excluding I/O):")
    print(f"  DeepLabV3:   {frame_idx / time_dl:.2f} fps")
    print(f"  Lite R-ASPP: {frame_idx / time_lr:.2f} fps")
```

```python
n_show = min(4, len(samples))
if n_show == 0:
    print("No frames captured for inline preview (empty video?)")
else:
    fig, axes = plt.subplots(n_show, 3, figsize=(12, 3.2 * n_show))
    if n_show == 1:
        axes = np.array([axes])
    for r in range(n_show):
        fr, pdl, plr = samples[r]
        axes[r, 0].imshow(fr)
        axes[r, 0].set_title("Frame (resized)")
        axes[r, 1].imshow(bgr_dl_over[:, :, ::-1])
        axes[r, 1].set_title("DeepLabV3 (Q4)")
        axes[r, 2].imshow(bgr_lr_over[:, :, ::-1])
        axes[r, 2].set_title("Lite R-ASPP (Q5)")
        for c in range(3):
            axes[r, c].axis("off")
    plt.suptitle("Sample frames: qualitative comparison for Question 6")
    plt.tight_layout()
    plt.show()
```

```
Wrote 171 frames to:
  /home/pc/projects/COSC6373/HW12/Part_B/outputs/hw12b_segmentation_deeplabv3_mo
bilenetv3.mp4
  /home/pc/projects/COSC6373/HW12/Part_B/outputs/hw12b_segmentation_lraspp_mobil
enetv3.mp4
  /home/pc/projects/COSC6373/HW12/Part_B/outputs/hw12b_segmentation_compare_orig
inal_deeplab_lraspp.mp4

Average Inference FPS (excluding I/O):
  DeepLabV3:   59.13 fps
  Lite R-ASPP: 73.14 fps
```

1.5 Question 6

Which model seems to perform better? (qualitative comparison via visual inspection)

Without ground-truth masks, the comparison is necessarily qualitative. I watched the full comparison export (`outputs/hw12b_segmentation_compare_original_deeplab_lraspp.mp4`) and the per-model videos. In most frames, DeepLabV3 (MobileNetV3) shows slightly more stable region boundaries and a bit less speckle inside large regions, while Lite R-ASPP is a little noisier or more fragmented, especially on thin structures and at class transitions. A lighter segmentation head is usually chosen for higher throughput, which often trades off some label consistency at boundaries, and that lines up with what I see here. On some clips the gap is small; the result still depends a lot on scene content, lighting, and how many of the 20 foreground classes appear in the frame. Now that we measure inference time, we can quantitatively confirm that Lite R-ASPP achieves higher frames-per-second than DeepLabV3, which illustrates the trade-off between segmentation stability

and processing speed.

1.6 Question 7

What is the main difference between DeepLabV3 and LR-ASPP? Please elaborate.

DeepLabV3 is a heavier segmentation design that uses atrous (dilated) convolutions and an Atrous Spatial Pyramid Pooling (ASPP) module to pool multi-scale context before forming dense predictions. The MobileNetV3-based variant in `torchvision` keeps that full DeepLabV3 head on top of a mobile backbone, trading some compute versus a ResNet-style backbone, but the overall role of ASPP is the same: strong multi-scale context.

Lite R-ASPP (Lite Reduced ASPP) is a compact segmentation head proposed with MobileNetV3. It reuses a reduced ASPP-style design so the model stays fast and small for mobile and edge use. In practice, both checkpoints used here are trained for the same 21-class Pascal VOC style outputs and the same pre-processing contract, but the LR-ASPP head is the option tuned for lower latency, while DeepLabV3 is the more expressive decoder among these `torchvision` options.

1.7 Acknowledgment

I used a GPT-5.3-Codex to help scaffold and organize this notebook.

Gemini-3.1 was used to check the result and validate conformity with the assignment outline.

All observations and responses were written by me, Adam Nelson-Archer.